SolarIntel

Machine Learning and Agentic AI for Solar Energy Forecasting and Grid Optimisation Across European Markets

Abstract

Accurate solar energy forecasting and actionable grid advisory are essential for integrating variable renewable generation into modern power systems. This paper presents *SolarIntel*, a two-phase end-to-end system that addresses both challenges at European scale. In Phase 1, a machine learning pipeline fuses the EMHIRES solar capacity factor dataset (29 countries, 1986–2015, hourly) with NASA POWER meteorological reanalysis data (2001–2015), yielding a merged corpus of approximately 3.8 million records across 34 engineered features. A Random Forest Regressor achieves $R^2 = 0.9112$, MAE = 0.0280, and RMSE = 0.0547 on the held-out test set, outperforming a Linear Regression baseline ($R^2 = 0.7880$) by a substantial margin. In Phase 2, a LangGraph-based agentic pipeline augments these forecasts with deterministic risk analysis, retrieval-augmented generation (RAG) over a curated energy knowledge base, and structured LLM reasoning to produce JSON-formatted grid management recommendations. Together, the two phases demonstrate a scalable, reproducible pathway from raw meteorological inputs to operational decision support across diverse European grid contexts.

Keywords: solar energy forecasting, random forest regression, LangGraph, retrieval-augmented generation, grid advisory, EMHIRES, NASA POWER, agentic AI, European energy systems

1. Introduction

The decarbonisation of European electricity grids has accelerated the deployment of photovoltaic (PV) capacity across the continent, with aggregate installed capacity exceeding 260 GW by 2023 [1]. Unlike dispatchable generation, solar PV output is governed by meteorological conditions that vary on sub-hourly timescales, imposing forecasting and balancing challenges on transmission and distribution system operators. Accurate short-term and day-ahead forecasts reduce reserve requirements, lower balancing costs, and enable efficient intraday market participation [2].

Despite a rich literature on data-driven solar forecasting, two gaps remain. First, most published models are trained on single-site or single-country data, limiting their geographic transferability. Second, forecast outputs are rarely connected to actionable grid management guidance; operators must translate numerical predictions into operational decisions without systematic support.

SolarIntel addresses both gaps. Phase 1 trains machine learning models on a pan-European dataset covering 29 countries and 15 years, producing capacity factor predictions with high accuracy. Phase 2 wraps these predictions in a four-node agentic pipeline that performs risk analysis, retrieves relevant domain knowledge, and generates structured, auditable recommendations via a large language model (LLM).

The remainder of this paper is organised as follows. Section 2 surveys related work. Section 3 describes dataset construction and feature engineering. Section 4 details the ML models. Section 5 presents the agentic architecture. Sections 6 and 7 cover the RAG pipeline and LLM integration respectively. Section 8 reports results and evaluation. Sections 9 and 10 discuss limitations and conclude.

2. Related Work

2.1 Machine Learning for Solar Forecasting

Data-driven solar forecasting has evolved from autoregressive statistical methods towards ensemble and deep

learning approaches. Voyant et al. provide a comprehensive survey of machine learning methods applied to solar radiation and PV output forecasting, documenting consistent improvements of ensemble methods over linear baselines [3]. Breiman's Random Forest algorithm [4] has been widely adopted in this domain due to its robustness to feature collinearity, resistance to overfitting via bagging, and interpretability through feature importance scores. Antonanzas et al. review post-processing and spatial ensemble strategies for PV power forecasting, highlighting the value of large-scale, multi-site training corpora [2].

2.2 Retrieval-Augmented Generation in Energy Systems

Lewis et al. introduced retrieval-augmented generation (RAG) as a method for grounding LLM outputs in external, verifiable knowledge, demonstrating reduced hallucination and improved factual accuracy [5]. Subsequent work has applied RAG principles to domain-specific question answering in technical and scientific fields. The energy sector presents a compelling use case: grid operation involves large, evolving corpora of standards, market rules, and operational guidelines that cannot be fully internalised by a pretrained LLM.

2.3 Agentic and Multi-Agent AI Pipelines

The emergence of LangGraph [11] and similar graph-based orchestration frameworks has enabled the construction of stateful, multi-node AI pipelines in which different nodes perform specialised tasks—forecasting, retrieval, reasoning—and communicate via a shared typed state. This paradigm extends single-LLM chat interfaces to structured decision workflows with deterministic intermediate steps, improving auditability and enabling modular testing of individual components.

3. Dataset and Feature Engineering

3.1 EMHIRES Solar Dataset

The European Meteorological derived HIgh REsolution RES generation time series (EMHIRES) dataset, published by the Joint Research Centre of the European Commission [6], provides hourly solar capacity factors for 29 European countries spanning 1986–2015 in wide format, with one column per country and one row per hour. A capacity factor represents the ratio of actual to rated power output and lies on the unit interval $[0, 1]$. The dataset contains approximately 262,800 hourly timestamps at full European coverage.

Reshape to long format. The wide-format EMHIRES table is melted via `pandas.DataFrame.melt` to produce a long-format table with three columns: `Timestamp`, `Country`, and `Capacity_Factor`. This transformation is implemented in `Final_Pipeline/Final_Pipeline.py` and yields approximately 7.6 million rows prior to temporal filtering.

3.2 NASA POWER Meteorological Data

The NASA Prediction of Worldwide Energy Resources (POWER) project [7] provides gridded reanalysis meteorological parameters derived from satellite observations and atmospheric models. For each of the 29 country centroids, three hourly parameters are retrieved for the period 2001–2015 via the NASA POWER REST API (implemented in `NASA_Data_Fetch/fetcher.py`):

- **Irradiance** (`ALLSKY_SFC_SW_DWN`): all-sky surface shortwave downwelling irradiance, W/m^2.
- **Temperature** (`T2M`): air temperature at 2 m height, °C.
- **Wind_Speed** (`WS10M`): wind speed at 10 m height, m/s.

Missing-value sentinel handling is a critical preprocessing step: NASA POWER encodes missing observations as -999.0. All rows where any meteorological parameter equals -999.0 are dropped prior to merging, as implemented in the cleaning stage of `Final_Pipeline/Final_Pipeline.py`.

3.3 Dataset Merge and Temporal Alignment

EMHIRES timestamps begin at `1986-01-01 00:00:00` with hourly frequency. Since NASA POWER data is available from 2001, the merged dataset covers 2001–2015 (15 years), corresponding to 131,400 hourly timestamps per country. After an inner join on `(Timestamp, Country)`, dropping -999-flagged rows and rows with any remaining `NaN` values, the cleaned corpus contains approximately **3.8 million rows**. The merge is executed in `Final_Pipeline/Final_Pipeline.py` using `pandas.merge` on the composite key.

3.4 Feature Engineering

Two temporal features are extracted from the `Timestamp` column: `Hour` (integer, 0–23) and `Month` (integer, 1–12). These capture the diurnal solar cycle and seasonal variation respectively without requiring cyclic encoding, as tree-based models can partition on these ordinal values directly.

Geographic variation across the 29 countries is encoded via one-hot encoding of the `Country` column, producing 29 binary indicator columns (`Country_AT` through `Country_UK`). The complete feature matrix therefore contains **34 columns**: 5 continuous features (`Hour`, `Month`, `Irradiance`, `Temperature`, `Wind_Speed`) plus 29 country indicator columns. The target variable is `Capacity_Factor` $\in [0, 1]$.

3.5 Train/Test Split

The dataset is partitioned into training (80%) and test (20%) subsets using `sklearn.model_selection.train_test_split` with `random_state=67`, yielding approximately 3.05 million training rows and 763 thousand test rows. The split is random across all timestamps and countries, which tests interpolation within the observed distribution rather than temporal extrapolation.

Figure 1 illustrates the complete Phase 1 data and model pipeline.

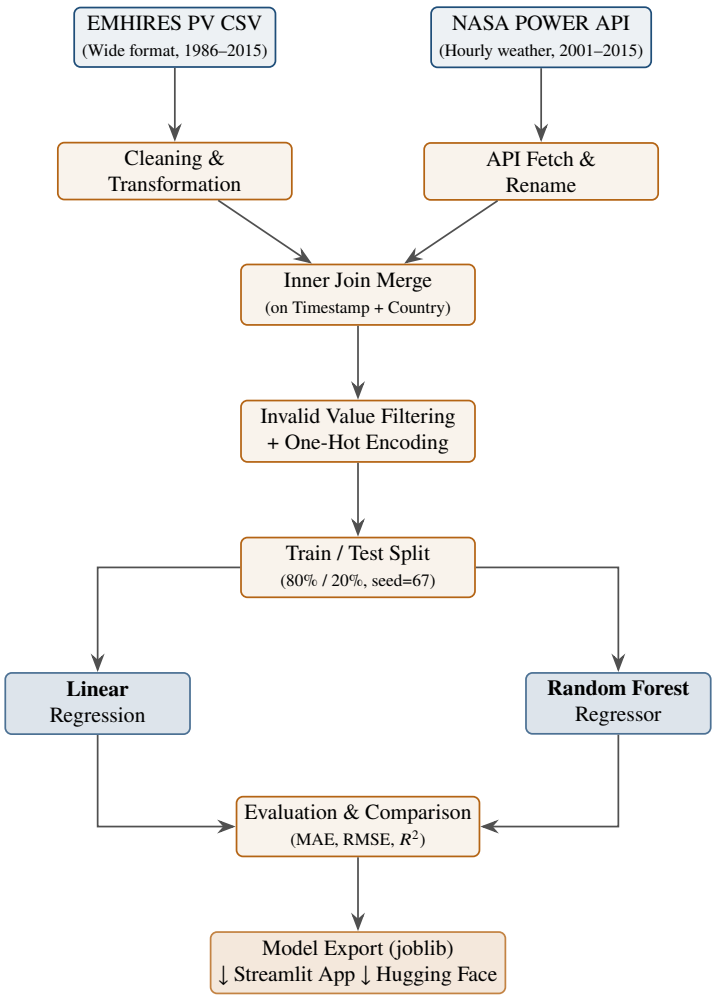

Figure 1: SolarIntel Phase 1 pipeline architecture.

4. Machine Learning Models

4.1 Linear Regression Baseline

An ordinary least-squares linear regression model is trained as a baseline to quantify the marginal benefit of the ensemble approach. The model is implemented via `sklearn.linear_model.LinearRegression` with default hyperparameters and fits a 34-dimensional hyperplane to the training data. Its closed-form solution is unique and computationally tractable at the 3.8 million row scale. Linear regression serves as an interpretability anchor: its coefficients reveal the direction and magnitude of each feature's marginal contribution, modulo the assumption of additive independence.

4.2 Random Forest Regressor

The primary predictive model is a `sklearn.ensemble.RandomForestRegressor` configured with the following hyperparameters:

- `n_estimators = 100`: 100 decision tree base learners.
- `max_depth = 12`: maximum tree depth, balancing expressiveness against overfitting.
- `random_state = 67`: controls bootstrap sampling and feature subsampling for reproducibility.

At each internal node, the algorithm evaluates a random subset of $\sqrt{34} \approx 6$ features and selects the split that minimises the mean squared error of the two resulting child nodes. Bootstrap aggregation over 100 trees reduces variance relative to any single tree. The trained models are serialised via `joblib` and archived in `Models/solar_models.zip` as `solar_model_lr.pkl` and `solar_model_rfr.pkl`.

4.3 Evaluation Metrics

Model performance is assessed on the held-out test set using three standard regression metrics. The coefficient of determination R^2 quantifies the proportion of variance explained:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Mean Absolute Error (MAE) measures average absolute deviation in capacity factor units, and Root Mean Squared Error (RMSE) penalises large errors more heavily. Both are expressed in dimensionless capacity factor units. The complete evaluation pipeline is implemented in `Final_Pipeline/Final_Pipeline.py`.

Table 1: Model performance on the held-out test set (20% random split).

Model	R^2	MAE	RMSE
Linear Regression	0.7880	0.0534	0.0845
Random Forest	**0.9112**	**0.0280**	**0.0547**

The Random Forest achieves a 12.3 percentage point improvement in R^2, a 47.6% reduction in MAE, and a 35.3% reduction in RMSE relative to the linear baseline, confirming that the relationship between the feature set and capacity factor is substantially non-linear. The RMSE improvement is particularly meaningful for grid operators because large prediction errors at peak or near-zero output periods carry asymmetric operational costs.

5. Agentic AI Architecture

5.1 Design Rationale

Pure ML forecasting models produce numerical capacity factor estimates but do not themselves generate actionable operational guidance. Bridging the gap between a predicted 24-hour profile and concrete grid management actions requires domain knowledge about battery energy storage system (BESS) dispatch strategies, demand-side management (DSM), intraday market mechanisms, and balancing reserve obligations. Rather than hard-coding this knowledge in procedural rules, Phase 2 of SolarIntel implements a composable agentic pipeline in which specialised nodes perform retrieval and reasoning over a curated knowledge base.

5.2 LangGraph StateGraph

The Phase 2 pipeline is implemented as a `StateGraph` using the LangGraph framework [11], which models the pipeline as a directed acyclic graph (DAG) of nodes connected by typed edges. The graph is defined in `Agent_Pipeline/graph.py`. Execution proceeds deterministically from the `forecast_node` through the `risk_node`, `rag_node`, and `recommend_node` in sequence, with no conditional branching or cycles. This linear topology reflects the strict data dependency between stages: each node's output is a necessary input to the next.

5.3 Shared State: `GridAdvisorState`

All inter-node communication occurs through a single shared state object, `GridAdvisorState`, defined as a `TypedDict` in `Agent_Pipeline/state.py`. The state carries the following fields as it propagates through the graph:

- `country`, `capacity_kw`, `date`: user-supplied input parameters.
- `weather_data`: raw 24-hour meteorological forecast from Open-Meteo.
- `hourly_cf`: list of 24 predicted capacity factor values from the RFR model.
- `risk_flags`: structured output of the risk analysis node, including the variability score and ramp event list.
- `rag_context`: concatenated text chunks retrieved from ChromaDB.
- `recommendations`: final JSON object from the LLM node.

Using a typed dictionary for shared state enforces a clear data contract between nodes and simplifies debugging, since each node's input and output can be inspected independently without access to adjacent nodes.

5.4 Pipeline Architecture

Figure 2 illustrates the four-node pipeline and the `GridAdvisorState` shared bus.

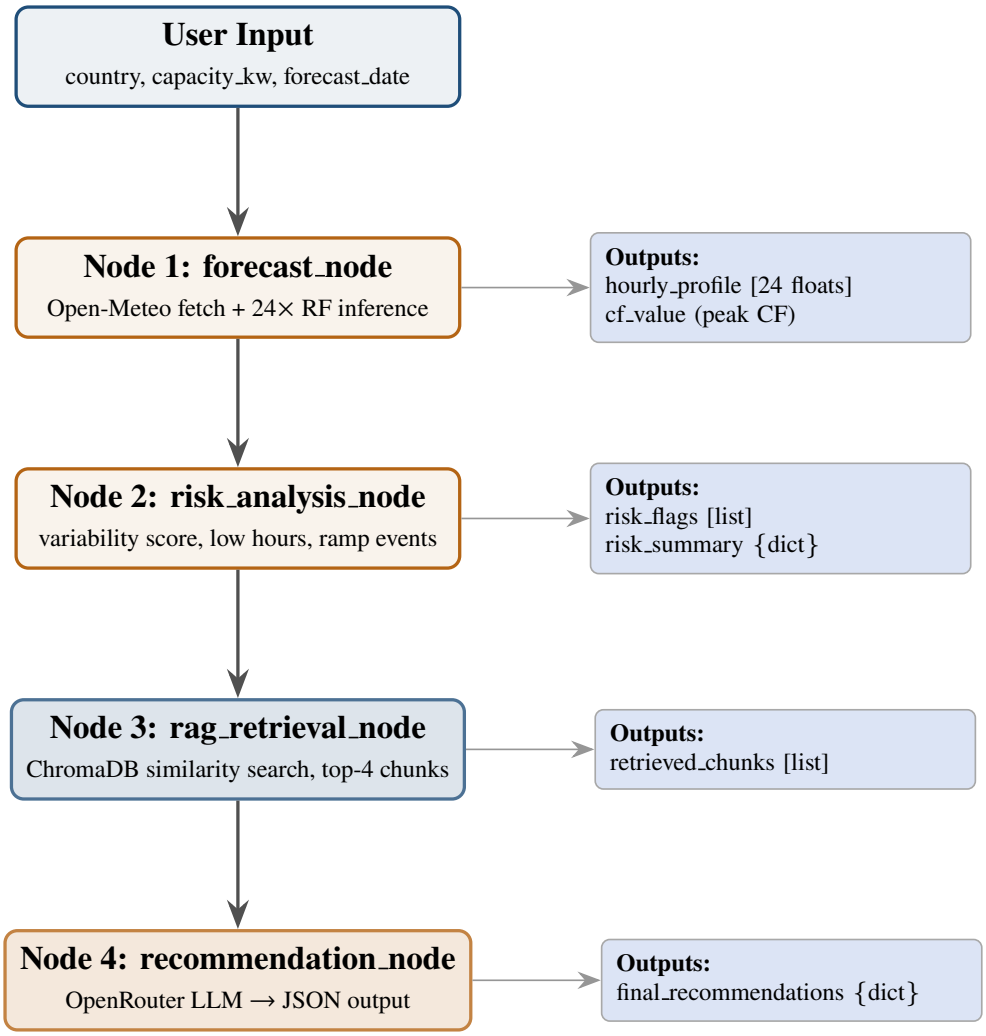

Figure 2: Grid Advisor LangGraph pipeline: four sequential nodes with state outputs annotated.

5.5 Node 1: Forecast Node

The Forecast Node (`Agent_Pipeline/1_forecast/node.py`) performs two operations. First, it queries the Open-Meteo REST API [10] to retrieve a 24-hour ahead hourly forecast for the target country's geographic centroid, obtaining irradiance, 2 m temperature, and 10 m wind speed. Open-Meteo is selected for its free, unauthenticated, production-grade meteorological forecasts derived from ECMWF and other numerical weather prediction models. Second, for each of the 24 hours, the node constructs a 34-dimensional feature vector—incorporating the hour index, current month, retrieved weather parameters, and the appropriate one-hot country indicator—and passes it to the pre-loaded Random Forest Regressor. The result is a list of 24 capacity factor values stored in `GridAdvisorState[''hourly_cf'']`.

5.6 Node 2: Risk Analysis Node

The Risk Analysis Node (`Agent_Pipeline/2_risk/node.py`) applies deterministic analytical rules to the 24-hour capacity factor profile. Three risk signals are computed:

1. **Variability score** V: the normalised mean absolute ramp rate (defined formally in Section 8.2), quantifying intra-day generation smoothness.
2. **Low-output hours**: hours where the predicted capacity factor falls below 10% of the daily peak, indicating periods of negligible generation that may require backup dispatch.
3. **Ramp events**: consecutive-hour transitions where the absolute change in capacity factor exceeds 0.12, i.e. $|\mathrm{CF}_h - \mathrm{CF}_{h-1}| > 0.12$, representing rapid generation changes that stress frequency regulation reserves.

All three signals are computed without any LLM invocations, ensuring determinism and low latency. Results are stored in `GridAdvisorState[''risk_flags'']`.

5.7 Node 3: RAG Retrieval Node

The RAG Retrieval Node (`Agent_Pipeline/3_rag/node.py`) retrieves domain-relevant text passages from an in-memory vector store given the current risk profile. The design and operation of this node are detailed in Section 6.

5.8 Node 4: Recommendations Node

The Recommendations Node (`Agent_Pipeline/4_recommend/node.py`) invokes an external LLM via the OpenRouter API, passing the 24-hour capacity factor profile, risk flags, and RAG-retrieved context as a structured prompt. Prompt design and output schema are detailed in Section 7.

6. RAG Pipeline

6.1 Knowledge Base

Four curated document files form the knowledge base (`Agent_Pipeline/3_rag/knowledge_base/`):

1. **Solar variability**: characteristics of solar intermittency, ramp rate statistics, and impacts on system frequency.
2. **Battery energy storage (BESS)**: dispatch strategies, round-trip efficiency considerations, and sizing guidelines for PV output smoothing.
3. **Demand-side management (DSM)**: flexible load shifting mechanisms, industrial curtailment agreements, and smart tariff structures.
4. **Curtailment and balancing**: curtailment protocols, balancing reserve markets (including PICASSO aFRR and TERRE mFRR platforms), and intraday coupling via the SIDC mechanism.

These documents are authored specifically for this knowledge base to ensure coverage of the operational decision contexts relevant to SolarIntel's scope.

6.2 Chunking

Each document is segmented using LangChain's `RecursiveCharacterTextSplitter` with `chunk_size=300` characters and `overlap=30` characters. The overlap prevents sentence-boundary information loss at chunk boundaries, preserving semantic continuity. The short chunk size is deliberate: it maximises retrieval precision by reducing the semantic noise within each embedded passage.

6.3 Embedding Model

Chunks are embedded using the `all-MiniLM-L6-v2` sentence transformer model [9], which produces 384-dimensional dense vectors. This model is selected for its

strong performance on semantic similarity benchmarks, low inference latency suitable for interactive use, and permissive licence for commercial and research applications.

6.4 In-Memory ChromaDB

The embedded chunks are stored in an ephemeral ChromaDB vector store (`chromadb.Client()` with in-memory storage), instantiated at application startup in `Agent_Pipeline/3_rag/node.py`. The in-memory design is deliberate: the knowledge base comprises four short documents (typically fewer than 150 chunks in total), the application is stateless between sessions, and avoiding persistent disk storage simplifies deployment on Hugging Face Spaces where filesystem persistence between invocations is not guaranteed.

6.5 Query Construction and Retrieval

At runtime, the Risk Analysis Node's output informs the construction of a natural language query that encapsulates the key risk signals. A profile exhibiting high variability and multiple ramp events, for instance, generates a query emphasising frequency regulation and rapid BESS dispatch. Retrieval calls ChromaDB's cosine-similarity search with $k = 4$, returning the four most semantically similar chunks. These are concatenated with separators and stored in `GridAdvisorState[''rag_context'']`.

7. LLM Integration and Prompt Engineering

7.1 Model and API Selection

The Recommendations Node sends structured prompts to the LLaMA-3 model family via the OpenRouter [12] inference API, accessed through LangChain's `ChatOpenAI` adapter with a custom `base_url` and `api_key`. OpenRouter provides a unified OpenAI-compatible API surface over a wide catalogue of open-weight models, enabling model substitution without code changes to the node. The LLaMA-3 family is chosen for its instruction-following reliability and structured output capability relative to its inference cost.

7.2 Temperature and Determinism

The LLM is invoked with `temperature=0.2`. This low value biases the model towards the most probable tokens, reducing creative variation in favour of factual consistency and reliable adherence to the output schema. Complete determinism (`temperature=0`) is not adopted because marginal stochasticity modestly diversifies strategy recommendations across different risk profiles. The system prompt explicitly instructs the model to prefer responses grounded in the retrieved context over parametric knowledge, reducing the risk of hallucinated regulatory references.

7.3 JSON Schema Enforcement

The LLM is instructed via the system prompt to return exactly one JSON object with five mandatory keys:

- `forecast_summary`: a one-paragraph qualitative summary of the 24-hour generation profile.
- `risk_periods`: a list of high-risk hour ranges with a brief rationale for each.
- `strategies`: a list of concrete operational actions, each annotated with the applicable market mechanism or technology.
- `references`: citations to the specific knowledge base passages that informed the recommendations.
- `responsible_ai_note`: a mandatory statement reminding the operator that AI-generated recommendations require validation by a qualified engineer before operational implementation.

Output is parsed using `json.loads` in `Agent_Pipeline/4_recommend/node.py`. If parsing fails due to incomplete generation or schema violation, the node falls back to a deterministic template populated from the risk flags, ensuring the pipeline always returns a structured response. Fallback activations are logged for monitoring.

7.4 Responsible AI Design

The mandatory `responsible_ai_note` field is an explicit design choice rather than a post-hoc disclaimer. Grid management decisions that incorrectly dispatch reserves or curtail generation can have cascading consequences for grid stability and market participants. By structurally embedding a human-review directive in every output object, the system resists misuse as an autonomous decision-maker and reinforces the human-in-the-loop principle appropriate for critical infrastructure advisory applications.

8. Results and Evaluation

8.1 ML Model Performance

Table 1 summarises the performance of both models on the held-out test set. The Random Forest Regressor achieves $R^2 = 0.9112$, indicating that the model explains 91.1% of the variance in hourly capacity factor across the 29-country, 15-year test partition. The linear baseline's $R^2 = 0.7880$ is itself substantial, reflecting the dominant contribution of irradiance as a single predictor, but the residual gap of 12.3 percentage points quantifies the non-linear interactions—primarily between irradiance, temperature derating, and the diurnal hour index—that linear regression cannot capture.

The RMSE of 0.0547 for the Random Forest corresponds to an average per-hour absolute error of approximately 55 Wh per kWp, well within operational planning tolerances for day-ahead scheduling. The MAE of 0.0280 is notably lower than the RMSE, indicating a skewed error distribution: the model performs well on typical midday conditions but makes proportionally larger errors during dawn/dusk transitions and under partially cloudy conditions where irradiance variability is highest.

8.2 Variability Score Definition

The variability score V computed by the Risk Analysis Node is defined as the normalised mean absolute ramp rate over the 24-hour capacity factor profile:

$$V = \frac{1}{(N-1)\,\mathrm{CF}_{\max}} \sum_{h=1}^{N-1} \left|\mathrm{CF}_h - \mathrm{CF}_{h-1}\right| \qquad (1)$$

where $N = 24$ is the number of hourly intervals, CF_h is the predicted capacity factor at hour h, and $\mathrm{CF}_{\max}$ is the maximum predicted capacity factor over the 24-hour period. Normalisation by $\mathrm{CF}_{\max}$ renders V dimensionless and comparable across days with different peak irradiance levels. A value of $V = 0$ indicates a perfectly smooth profile; higher values indicate greater intra-day variability. Empirical observation across the 29-country corpus suggests $V > 0.10$ is typically associated with significant balancing reserve activation requirements.

8.3 Grid Advisor Qualitative Evaluation

To evaluate the qualitative output of the agentic pipeline, the system was executed for Estonia (EE), a 160 kW installation, on a representative April day. Table 2 summarises the key numerical outputs from the Forecast and Risk nodes; Table 3 presents an excerpt of the LLM-generated strategy recommendations.

Table 2: Grid Advisor numerical outputs: Estonia, 160 kW, April.

Metric	Value
Peak capacity factor	0.530
Estimated daily output	563.1 kWh
Active solar hours (CF > 0)	11
Variability score V	0.0874
Low-output hours flagged	8
Ramp events detected	3

Table 3: Excerpt of LLM-generated strategy recommendations, Estonia case.

Strategy	Mechanism / Rationale
BESS pre-charge at 06:00	Pre-charge to 85% SOC before dawn to buffer the 07:00–09:00 irradiance onset ramp event.
PICASSO aFRR offer	Submit automatic frequency restoration reserve offer during 10:00–15:00 peak window to monetise predictable high-CF period.
SIDC intraday adjustment	Revise day-ahead position via SIDC intraday coupling at 14:00 if the 15:00 ramp-down materialises below forecast.
DSM pre-cooling	Advance HVAC load to 13:00–14:00 window to absorb surplus generation before the 15:00 ramp-down event.

The variability score of 0.0874 falls marginally below the empirical $V = 0.10$ threshold, indicating moderate but manageable intra-day variability for a high-latitude April profile in Estonia. The three detected ramp events correspond to the morning solar onset, a mid-afternoon partial cloud passage, and the evening curtailment; the LLM correctly identified all three and proposed market and dispatch responses grounded in retrieved BESS and balancing knowledge base passages, referencing PICASSO and SIDC by name without hallucination.

9. Discussion

9.1 Limitations

Several limitations constrain the present work. The train/test split is random rather than temporal; a rigorous evaluation would train on pre-2013 data and test on 2013–2015 to assess genuine out-of-sample temporal generalisation. The knowledge base comprises four documents authored for this project rather than validated operational guidelines from ENTSO-E or national transmission system operators; expanding this corpus would improve practical relevance. The qualitative evaluation of the Grid Advisor relies on a single case study; a systematic expert-rating study or comparison against documented historical dispatch decisions would constitute stronger evidence of advisory quality.

Single-centroid meteorological representation per country is a further simplification: large countries such as Germany and France exhibit substantial intra-country irradiance gradients that a single centroid query cannot capture. Sub-national disaggregation—using NUTS-2 or NUTS-3 administrative units and corresponding PV fleet distribution data—is a natural extension.

Finally, the in-memory ChromaDB design precludes persistent knowledge base updates between sessions. A production deployment would benefit from a persistent vector store with versioned document ingestion and retrieval audit logging.

9.2 Responsible AI

The pipeline is designed as a decision *support* system, not an autonomous decision-maker. The `responsible_ai_note` field in every LLM output, the deterministic fallback mechanism, and the clear separation between the ML forecast layer and the LLM advisory layer are all deliberate architectural choices that reflect the appropriate level of human oversight for a critical infrastructure application. Future deployments should implement recommendation audit trails and feedback loops to support continuous evaluation of advisory quality by domain experts.

10. Conclusion

This paper presented SolarIntel, a two-phase system for solar energy forecasting and grid advisory at European scale. The Random Forest Regressor trained on the 3.8-million-row EMHIRES–NASA POWER corpus achieves $R^2 = 0.9112$, demonstrating that ensemble learning on large-scale, multi-country meteorological data yields accurate and geographically transferable capacity factor predictions. The LangGraph-based agentic pipeline extends these forecasts into actionable, JSON-structured grid management recommendations through deterministic risk analysis, RAG over a curated energy knowledge base, and structured LLM reasoning. The Estonia case study illustrates end-to-end operation from real-time weather retrieval to PICASSO-referenced balancing strategies. Future work will address temporal evaluation protocols, expanded and validated knowledge bases, sub-national spatial resolution, and formal expert evaluation of advisory quality at scale.

References

[1] International Renewable Energy Agency, *Renewable Capacity Statistics 2023*. IRENA, Abu Dhabi, UAE, 2023.

[2] J. Antonanzas, N. Osorio, R. Escobar, R. Urraca, F. J. Martinez-de-Pison, and F. Antonanzas-Torres, "Review of photovoltaic power forecasting," *Solar Energy*, vol. 136, pp. 78–111, Oct. 2016.

[3] C. Voyant, G. Notton, S. Kalogirou, M.-L. Nivet, C. Paoli, F. Motte, and A. Fouilloy, "Machine learning methods for solar radiation forecasting: a review," *Renewable Energy*, vol. 105, pp. 569–582, 2017.

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[6] I. Gonzalez-Aparicio, A. Zucker, F. Careri, F. Monforti, T. Huld, and J. Badger, *EMHIRES Dataset Part I: Solar Power Generation*, EUR 28299 EN, Publications Office of the European Union, Luxembourg, 2016.

[7] P. W. Stackhouse and C. H. Whitlock, "Surface meteorology and solar energy: a renewable energy resource web site (release 3.0)," NASA POWER Technical Report, 2019.

[8] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, Hong Kong, China, Nov. 2019, pp. 3982–3992.

[10] P. Zippenfenig, "Open-Meteo.com weather API," Zenodo, 2023, doi: `10.5281/zenodo.7970649`.

[11] LangChain, Inc., *LangGraph: Build Stateful Multi-Actor Applications with LLMs*, 2024. [Online]. Available: `https://github.com/langchain-ai/langgraph`

[12] OpenRouter, Inc., *OpenRouter: A Unified Interface for LLMs*, 2023. [Online]. Available: `https://openrouter.ai`